

Cosmos Policy

推理流水线

直接策略推理流程 | RoboCasa 数据集

NVIDIA Cosmos Policy — 基于视频基础模型的机器人控制策略

每页分析一个模块：输入维度 -> 功能原理 -> 输出维度 -> 代码位置



流水线总览

Pipeline Overview



RoboCasa 配置说明

state_t=11 帧观测历史 | chunk_size=32 步动作预测 | action_dim=7 自由度 | proprio_dim=9 本体感知维度
3个相机视角 (agentview, eye_in_hand, robot0_robotface)
总隐帧数 = 11帧视频 + 8帧动作 + 2帧本体感知 = 21帧
每个隐帧形状: (1, 16, 28, 28), 输入 224x224 经过 8x 空间压缩

CODE: cosmos_policy/constants.py | cosmos_policy/experiments/robot/cosmos_utils.py:61-85 | cosmos_policy/config/experiment/cosmos_policy_experiment_configs.py

模块 1: 观测数据采集

Observation Data Collection

输入 INPUT

来自 RoboCasa 仿真环境的原始传感器数据:

- 3路相机图像: 各 (B, 3, 224, 224) [agentview, eye_in_hand, robotface]
- 本体感知状态: (B, 9) [关节位置(7) + 夹爪(2)]
- 任务语言指令: 字符串, 如 'pick up the red block'

原理 FUNCTION

从机器人环境采集多模态观测数据。

1. 从3个标定相机捕获 224x224 RGB 图像。
2. 读取机器人关节位置 (7自由度) + 夹爪状态 (2值) = 9维本体感知。
3. 接收自然语言任务描述。
4. 堆叠历史帧: `state_t=11` 帧观测历史。
5. 图像归一化到 `[-1, 1]` 范围, 本体感知按维度归一化。

输出 OUTPUT

处理后的观测张量:

- `images_list`: 长度为3的列表, 每个 (B, 3, 224, 224)
- `proprio`: (B, 9) 归一化的关节+夹爪状态
- `task_text`: 字符串, 回合的语言指令

数据格式

RoboCasa 数据集: HDF5 格式, 包含 `observations/qpos/gripper_state/actions` 字段
`state_t=11` 表示堆叠11帧历史观测作为上下文

CODE: `cosmos_policy/experiments/robot/cosmos_utils.py:851 get_action() | prepare_data_batch()`

模块 2: VAE 编码器 (视频分词)

WAN 2.1 VAE Encoder

输入 INPUT

- 原始视频帧: (B, 3, T, H, W)
- 每个相机视角: (B, 3, state_t=11, 224, 224)
 - 像素范围: [-1, 1] (已归一化)
 - 3路独立相机流分别编码

原理 FUNCTION

- WAN 2.1 视频 VAE 编码器将像素空间视频压缩为紧凑的隐空间。
1. 空间维度: 通过2D卷积实现8倍下采样 (224->112->56->28)。
 2. 时间维度: 通过3D因果卷积实现4倍下采样 (每4帧为一组)。
 3. 通道维度: 3 (RGB) -> 16 (隐变量通道)。
 4. KL正则化保持隐变量在良好分布内, 便于扩散模型处理。
 5. 每个相机视角独立编码。
 6. state_t=11 帧 -> ceil(11/4)=3 个时间隐帧/视角。

输出 OUTPUT

- 隐变量表示: (B, z_dim=16, T', H', W')
- 每个相机: (B, 16, 3, 28, 28) [T'=11/4≈3, 填充至3个时间帧]
 - 空间压缩: $224/8 = 28$
 - 时间压缩: 4x (4个像素帧 -> 1个隐帧)
 - 隐空间: 连续的, 由 KL 正则化 VAE 学习

压缩比

总压缩比: 8x 空间 * 4x 时间 = 32倍
每个 224x224 RGB 帧 -> 1x28x28x16 隐变量 = 12,544 个值 (对比 150,528 像素)

CODE: cosmos_policy/tokenizers/wan2pt1.py:197 class CosmosPolicyWanVAE | :303 encode() | :44 class WanVAE_ | :51 encode()

模块 3: 隐帧布局 (隐变量帧注入)

Latent Frame Injection & Sequence Layout

输入 INPUT

观测隐变量 + 动作/本体感知/价值 元数据:

- 3路相机隐变量: 各 (B, 16, 3, 28, 28)
- 动作块: (B, 32, 7) = 224个值, 推理时全零
- 本体感知状态: (B, 9) 个值
- 价值占位符: 标量 (用于规划模式)

原理 FUNCTION

核心创新: 将所有模态编码到统一的隐帧序列中。

1. 动作隐变量注入: 224个动作值 (32x7) 重复填充到8个隐帧中。
每帧有 $16 \times 28 \times 28 = 12,544$ 个槽位, 每帧前 $224/8=28$ 个值被设置 (重复填充)。
2. 本体感知注入: 9个值以同样方式重复填充到2个隐帧中。
3. 时间复制: WAN VAE 将4个像素帧映射为1个隐帧, 因此每个概念
需要4张相同图像才能产生1个隐帧 (训练时处理)。
4. 推理时: 动作/本体感知帧初始化为噪声 (等待去噪)。

完整隐变量序列: (B, 16, total_frames, 28, 28)

输出 OUTPUT

RoboCasa 布局 (共21帧):

帧 0-2: 相机1 隐变量 (3帧)

帧 3-4: 相机2 隐变量 (2帧)

帧 5-6: 相机3 隐变量 (2帧)

帧 7-14: 动作 隐变量 (8帧, $32 \text{步} \times 7 \text{维}$ 映射)

帧 15-16: 本体感知 隐变量 (2帧, 9维 映射)

帧 17-20: 未来状态预测 (4帧)

核心思想

将机器人动作视为'伪视频帧' -- 同一个扩散过程同时去噪视频和预测动作。

这是核心洞察: 通过共享隐空间实现统一的多模态生成。

CODE: cosmos_policy/models/policy_text2world_model.py:45 replace_latent_with_action_chunk() | :112 replace_latent_with_proprio()

模块 4: 条件掩码

Condition Mask

输入 INPUT

- 完整隐变量序列: (B, 16, 21, 28, 28)
- 视频观测帧: 已知/干净的 (来自真实观测的编码)
 - 动作/本体感知/未来帧: 推理时未知 (需要去噪生成)

原理 FUNCTION

- 二值掩码告诉模型哪些帧是'已知条件', 哪些是'待预测目标'。
1. 观测帧 (mask=1): 来自 VAE 编码器的干净隐变量。
扩散过程中, 不会对这些帧添加噪声。
 2. 动作/本体感知/未来帧 (mask=0): 去噪的目标。
训练时添加噪声, 模型学习如何去噪。
 3. 推理时: 观测帧保持固定, 动作帧从纯噪声开始,
经迭代去噪产生动作预测。
 4. 支持灵活的条件控制: 不同帧类型可以独立设置干净或含噪。

输出 OUTPUT

- 二值条件掩码: (B, 1, 21, 28, 28) 或 (21,) 广播
- mask=1 对应干净观测帧 (帧 0-6: 相机隐变量)
 - mask=0 对应去噪目标帧 (帧 7-20: 动作/本体感知/未来)
 - 控制哪些帧保持固定, 哪些由扩散模型预测

CODE: cosmos_policy/models/policy_video2world_model.py:100 get_data_and_condition() | policy_text2world_model.py:246 compute_loss_with_epsilon_and_sigma()

模块 5: 文本条件编码

Text Condition Encoding

输入 INPUT

任务指令字符串: 如 'pick up the red block and place it on the green target'

- 来自 RoboCasa 任务规格的原始文本
- 每个回合一条指令 (序列中所有帧共享)

原理 FUNCTION

将语言指令编码为稠密嵌入, 用于交叉注意力。

1. 使用 T5 分词器对指令进行分词。
2. 通过冻结的 T5-XXL 编码器 (46亿参数)。
3. 投影到交叉注意力维度 (1024)。
4. GeneralConditioner 处理 dropout (无分类器引导训练)。推理时: dropout=0, 始终以文本为条件。
5. Text2WorldCondition 扩展 BaseCondition, 在文本之外增加视频条件上下文。

输出 OUTPUT

交叉注意力嵌入: (B, seq_len, crossattn_dim)

- seq_len ~20-77 个token (T5 分词器输出)
- crossattn_dim = 1024 (匹配 DiT 隐藏维度)
- 在 DiT 网络的每个交叉注意力层中使用

无分类器引导 (CFG)

训练时随机丢弃文本 (10-20%)。推理时 guidance_scale 控制文本条件强度:

$\text{predicted} = \text{uncond} + \text{scale} * (\text{cond} - \text{uncond})$

scale 越大, 模型越严格遵循文本指令。

CODE: cosmos_policy/conditioner.py:86 class Text2WorldCondition | :112 class GeneralConditioner

模块 6: 噪声调度 (训练阶段)

Hybrid EDM SDE Noise Schedule

输入 INPUT

干净隐变量 x_0 : (B, 16, 21, 28, 28) [仅去噪目标帧参与]
- 训练时: 从混合分布中采样噪声等级 σ

原理 FUNCTION

混合 EDM 噪声调度, 确保训练稳定性。

- 70% 概率: 从对数正态分布 $N(P_mean, P_std^2)$ 采样 σ 。
 $\log(\sigma) \sim N(-1.2, 1.2^2)$ 。偏向中等噪声等级。
- 30% 概率: 从 $[\sigma_min, \sigma_max]$ 均匀采样 σ 。
确保覆盖极端噪声等级。
- 推理时: σ 从 σ_max 到 σ_min 确定性递减。
- EDM 预处理缩放网络输入/输出:
 $c_skip(\sigma), c_out(\sigma), c_in(\sigma), c_noise(\sigma)$ 函数。

输出 OUTPUT

含噪隐变量 $x_sigma = x_0 + \sigma * \epsilon$:
(B, 16, 21, 28, 28) [$\epsilon \sim N(0, I)$]
- σ 值按批次从混合分布采样
- 仅 $mask=0$ 的帧添加噪声; $mask=1$ 帧保持干净

EDM 预处理公式

$F_denoise(x, \sigma) = c_skip * x + c_out * Net(c_in * x, c_noise(\sigma))$
 $c_in = 1/(\sigma^2+1)^{0.5} \mid c_out = \sigma/(\sigma^2+1)^{0.5} \mid c_skip = 1 \mid c_noise = 0.25*\log(\sigma)$

CODE: cosmos_policy/modules/hybrid_edm_sde.py:22 class HybridEDMSDE | :48 sample_t()

模块 7: DiT 网络 (扩散 Transformer)

Diffusion Transformer — Core Network

输入 INPUT

含噪隐变量: (B, 16, 21, 28, 28) 经 c_in 缩放
时间步嵌入: sigma -> c_noise -> 正弦位置嵌入
交叉注意力上下文: (B, token_len, 1024) 来自文本编码器
条件掩码: (B, 1, 21, 28, 28)

原理 FUNCTION

- 20亿参数的扩散 Transformer，采用自注意力 + 交叉注意力架构。
1. 补丁嵌入: 将 (16, 28, 28) 空间展平为补丁 token 序列。
 2. 位置编码: 3D RoPE (空间 + 时间位置)。
 3. 28个 DiT 块，每个包含:
 - AdaLN (自适应层归一化): 以时间步 sigma 为条件。
 - 自注意力: 跨所有帧位置进行注意力 (时间 + 空间)。
 - 交叉注意力: 关注来自 T5 的文本嵌入。
 - 前馈网络: GPT-2 风格 MLP + GELU。
 4. 最终层: 线性投影回 (16, 28, 28) 每帧。
 5. 总参数量 20亿，bf16 混合精度。

输出 OUTPUT

去噪预测 F_theta: (B, 16, 21, 28, 28)
经过 EDM 输出缩放: $\hat{x} = c_{\text{skip}} * x_{\text{noisy}} + c_{\text{out}} * F_{\text{theta}}$
仅 mask=0 的帧包含有意义的预测

架构流

输入: (B, 16, T=21, 28, 28) -> PatchEmbed -> T*784个维度1024的token
-> 28x DiT块 (SelfAttn + CrossAttn + FFN) -> FinalLayer -> (B, 16, T, 28, 28)

CODE: cosmos_policy/_src/predict2/networks/minimal_v4_dit.py:1386 class MiniTrainDiT | :1713 forward() | :389 class Attention | :583 class I2VCrossAttention | :1049 class FinalLayer

模块 8: EDM 预处理

EDM Preconditioning

输入 INPUT

原始含噪隐变量 x_{sigma} : (B, 16, 21, 28, 28)
 sigma 值: 标量或 (B,) 张量
DiT 网络原始输出 F_{theta} : (B, 16, 21, 28, 28)

原理 FUNCTION

EDM (Elucidating Design Space) 预处理稳定扩散训练。
数学公式:
输入缩放: $c_{\text{in}}(\text{sigma}) = 1 / \sqrt{\text{sigma}^2 + 1}$
输出缩放: $c_{\text{out}}(\text{sigma}) = \text{sigma} / \sqrt{\text{sigma}^2 + 1}$
跳跃缩放: $c_{\text{skip}}(\text{sigma}) = 1$ (单位跳跃连接)
噪声映射: $c_{\text{noise}}(\text{sigma}) = 0.25 * \ln(\text{sigma})$

网络看到单位方差输入。输出组合跳跃连接 (保留信号)
与网络预测 (添加修正)。关键洞察: 当 $\text{sigma} \rightarrow 0$ 时, $c_{\text{out}} \rightarrow 0$, 输出自然逼近干净的 x_0 。

输出 OUTPUT

DiT 预处理输入: $c_{\text{in}}(\text{sigma}) * x_{\text{sigma}}$
最终去噪估计: $c_{\text{skip}}(\text{sigma}) * x_{\text{sigma}} + c_{\text{out}}(\text{sigma}) * F_{\text{theta}}$
 x_0 估计: (B, 16, 21, 28, 28), 干净观测帧不受影响

CODE: cosmos_policy/models/policy_text2world_model.py:310 compute_loss_with_epsilon_and_sigma() | minimal_v4_dit.py:860 Timesteps | :884 TimestepEmbedding

模块 9: 迭代采样 (去噪循环)

Iterative Denoising Loop

输入 INPUT

纯噪声 x_T : (B, 16, 21, 28, 28), $\sigma = \sigma_{\max}$

- 仅 $\text{mask}=0$ 的帧为噪声; $\text{mask}=1$ 的帧为干净观测隐变量
- num_steps : 去噪步数 (RoboCasa 通常35步)

原理 FUNCTION

从纯噪声迭代去噪到干净隐变量预测。

1. 将 $\text{mask}=0$ 的帧初始化为 $\sigma_{\max}$ 处的噪声 ($\text{mask}=1$ 帧保持干净)。
2. 对每一步 $t = T, T-1, \dots, 1$:
 - a. 计算当前 σ_t 和下一步 σ_{t-1} 。
 - b. 通过去噪函数 (EDM + DiT) 获取 x_0 预测。
 - c. 执行确定性 ODE 步: $x_{t-1} = x_t + (\sigma_{t-1} - \sigma_t) * \text{score}$ 。
 - d. 特殊情况: $\text{num_steps}=1$ 时直接使用 x_0 预测 (单步)。
3. 最终输出: 完全去噪的隐变量序列。
4. CosmosPolicySampler 支持 $\text{sample_clean}=\text{True}$ 进行掩码感知采样。

输出 OUTPUT

去噪后 x_0 : (B, 16, 21, 28, 28)

- $\text{mask}=0$ 的帧现在包含预测的动作/本体感知/未来隐变量
- $\text{mask}=1$ 的观测帧在整个采样过程中保持不变

调度策略

Sigma 调度: 从 $\sigma_{\max}$ (~80) 到 $\sigma_{\min}$ (~0.002) 的几何级数

ODE 求解器: Euler 或 RK4。默认: Euler 35步 (RoboCasa 推理)。

CODE: cosmos_policy/modules/cosmos_sampler.py:40 class CosmosPolicySampler | :96 _forward_impl()

模块 10: VAE 解码器

WAN 2.1 VAE Decoder

输入 INPUT

- 去噪后的隐变量序列: (B, 16, 21, 28, 28)
- 动作/本体感知/未来帧需要解码
 - 实际操作: 可选解码, 主要用于可视化验证

原理 FUNCTION

- WAN 2.1 VAE 解码器从隐变量重建像素视频。
1. 时间上采样: $T \rightarrow T \times 4$ (4倍时间压缩的逆过程)。
 2. 空间上采样: $28 \rightarrow 224$ (8倍空间压缩的逆过程)。
 3. 通道投影: $16 \rightarrow 3$ (隐变量 $\rightarrow$ RGB)。
 4. 使用3D转置卷积实现时空上采样。

重要提示: 动作提取时不需要解码为像素。
动作直接从隐帧中读取 (下一个模块)。
VAE 解码是可选的, 主要用于调试/可视化。

输出 OUTPUT

- 解码后像素空间视频: (B, 3, $T \times 4$, $H \times 8$, $W \times 8$)
- 每个动作帧: (B, 3, 32, 224, 224) -- 但不用于动作提取!
 - 动作值直接从隐变量提取, 不需要解码为像素
 - VAE 解码仅用于可视化/调试

关键洞察

动作存在于隐空间, 而非像素空间。
'伪视频帧'技巧意味着动作以隐变量值存储, 不需要渲染为像素。解码对机器人控制并非必要。

CODE: cosmos_policy/tokenizers/wan2pt1.py:341 decode() | :197 class CosmosPolicyWanVAE | :518 decode()

模块 11: 动作提取

Action Extraction from Latent Frames

输入 INPUT

去噪后的隐变量序列: (B, 16, 21, 28, 28)

- 动作隐帧位于索引 7-14 (RoboCasa 共8帧)
- 每帧: $16 * 28 * 28 = 12,544$ 个值

原理 FUNCTION

逆向隐变量注入过程, 恢复动作值。

1. 按索引选择动作隐帧。
2. 对每个动作帧: 展平 (16, 28, 28) -> 12,544 个值。
3. 每帧取前28个值 ($224\text{总值} / 8\text{帧} = 28$)。
4. 重塑为 (chunk_size=32, action_dim=7)。
5. 反归一化: 乘以 action_std, 加上 action_mean。
6. 裁剪到有效机器人指令范围。

输出 OUTPUT

动作块: (B, chunk_size=32, action_dim=7)

- 32个动作步, 每个7自由度 (x,y,z,rx,ry,rz,gripper)
- 从8个隐帧中提取224个值 (每帧28个值)
- 值经过反归一化到机器人指令范围

CODE: cosmos_policy/models/policy_text2world_model.py:45 replace_latent_with_action_chunk() | cosmos_utils.py:851 get_action() | :61 get_latent_indices_from_model_config()

模块 12: 本体感知提取

Proprioceptive State Extraction

输入 INPUT

去噪后的隐变量序列: (B, 16, 21, 28, 28)

- 本体感知隐帧位于索引 15-16 (RoboCasa 共2帧)
- 每帧: 12,544 个值, 但仅使用前约5个值

原理 FUNCTION

从隐帧中提取本体感知预测 (与动作提取方法相同)。

1. 按索引选择本体感知隐帧。
2. 展平每帧, 取前约5个值/帧。
3. 重塑为 `proprio_dim=9`。
4. 反归一化到物理关节/夹爪范围。
5. 模型作为去噪的副产物预测未来本体感知。

注意: 本体感知预测是辅助输出 -- 主要输出是动作块。
但它帮助模型保持时间一致性。

输出 OUTPUT

预测的本体感知: (B, 2, `proprio_dim=9`) 或 (B, 9)

- 预测的未来本体感知状态
- 可用于规划或状态预测验证
- 9个值: 7个关节位置 + 2个夹爪状态

CODE: `cosmos_policy/models/policy_text2world_model.py:112 replace_latent_with_proprio()` | `cosmos_utils.py:61 get_latent_indices_from_model_config()`

模块 13: 机器人执行

Robot Action Execution

输入 INPUT

- 动作块: (B, 32, 7) -- 32个动作步，每步7自由度
- 7自由度机械臂的绝对关节位置 + 夹爪
 - 已反归一化到机器人关节指令范围
 - 以开环方式预测

原理 FUNCTION

将预测的动作块转换为机器人关节指令。

1. 动作空间: 绝对关节位置 (非增量动作)。
2. 反归一化: $\text{action_raw} * \text{action_std} + \text{action_mean}$ 。
3. 裁剪到关节限位。
4. 以控制频率发送给底层 PD 控制器。
5. 夹爪指令: 连续值映射到开/合。

动作块提供32步未来控制，但仅执行其中一部分 (见开环控制页)。

输出 OUTPUT

发送给机器人控制器的关节指令:

- 每个时间步的7自由度关节位置目标
- 以控制频率执行 (如20Hz)
- 仅执行前 `execute_steps` 个动作后重新规划

CODE: `cosmos_policy/experiments/robot/cosmos_utils.py:851 get_action()` | RoboCasa 环境封装器

模块 14: 开环控制策略

Open-Loop Control Strategy

输入 INPUT

预测的动作块: (32, 7) -- 完整轨迹预测

- 一次前向传播预测32步未来控制
- 仅执行其中一部分后重新查询模型

原理 FUNCTION

时间动作分块与部分执行策略。

1. 模型在一次前向传播中预测 `chunk_size=32` 步未来动作。
2. 仅实际执行前 `execute_steps` 步 (如4或8步)。
3. 执行后采集新观测。
4. 用更新后的观测历史重新查询模型。
5. 优势:
 - 降低推理频率 (每4-8步推理一次, 而非每步)。
 - 块内提供时间一致性。
 - 每次重查询允许纠偏。
 - 权衡: `execute_steps` 越多 = 速度越快但反应性越差。

输出 OUTPUT

实际执行的动作: (`execute_steps`, 7) -- 通常4-8步

- 执行动作块中前 `execute_steps` 个动作
- 然后采集新观测, 重新查询模型
- 块中剩余动作被丢弃

权衡分析

`execute_steps=4`: 反应更灵敏, 推理成本更高, 纠偏更平滑

`execute_steps=8`: 反应较慢, 推理成本更低, 更坚持原计划

RoboCasa 默认: 通常4-8步

CODE: `cosmos_policy/experiments/robot/cosmos_utils.py:851 get_action()` | 实验配置: `execute_steps` 参数

模块 15: Best-of-N 规划

Best-of-N Planning

输入 INPUT N个候选动作块: $N \times (32, 7)$

- 通过运行N次独立去噪过程生成
- 每次从不同随机噪声开始 -> 不同的动作提议
- 通常 $N=4$ 或 $N=8$

原理 FUNCTION 生成多个动作候选, 按预测价值排序选择。

1. 独立运行N次去噪 (不同噪声种子)。
2. 对每个候选: 使用价值头预测其期望回报。
3. 选择具有最高预测价值的候选。
4. 这是一种简单但有效的测试时计算扩展方法。
5. 候选越多 = 选择越好, 但推理时间越长。
6. 类似语言模型中的采样-排序策略。

这是规划变体。对于直接策略 (无规划), $N=1$ 。

输出 OUTPUT 最优动作块: $(32, 7)$

- 基于价值预测从N个候选中选择
- 选择具有最高预测价值的候选
- 其余 $N-1$ 个候选被丢弃

价值头 模型还预测一个价值隐帧 (1个标量)。
该价值预测给定动作序列的期望回报。
用作 Best-of-N 选择的评分函数。

CODE: cosmos_policy/models/policy_video2world_model.py:569 get_x0_fn_from_batch() | cosmos_utils.py:851 get_action()

模块 16: 价值预测

Value Function Prediction

输入 INPUT

去噪后的隐变量序列: (B, 16, 21, 28, 28)

- 在特定索引处包含一个价值隐帧
- 价值以标量形式编码在一个隐帧中 (类似动作/本体感知)

原理 FUNCTION

预测所提议动作轨迹的期望回报。

1. 专用隐帧存储价值预测。
2. 价值的编码方式与动作类似: 标量重复填充到隐帧中。
3. 去噪后, 提取价值 (注入的逆过程)。
4. 训练时: `value_loss` 与 `action_loss` 分开掩码处理。
5. 可通过 `mask_value_prediction_loss_for_policy_prediction` 禁用。
6. 价值预测无需单独的 Critic 网络即可实现规划。

输出 OUTPUT

预测价值: 标量 (B, 1)

- 预测动作轨迹的期望回报
- 用于 **Best-of-N** 规划中的排序
- 价值越高 = 预测结果越好

CODE: `cosmos_policy/models/policy_text2world_model.py:186–210` (价值配置字段) | `:328` (`value_function_return`)

模块 17: 多视角融合

Multi-View Camera Fusion

输入 INPUT 3路独立相机流，各自独立编码：

- agentview (俯视/正面): (B, 16, 3, 28, 28)
- eye_in_hand (腕部): (B, 16, 2, 28, 28)
- robotface (侧面): (B, 16, 2, 28, 28)
- 每路捕获场景的不同空间信息

原理 FUNCTION 将多个相机视角融合到单一隐变量序列中。

1. 每个相机视角独立进行 VAE 编码。
2. 所有相机隐变量沿时间轴拼接。
3. DiT 的 3D RoPE 位置编码区分：
 - 每个视角内的空间位置 (28x28 网格)。
 - 帧间的时间位置。
 - 无需显式视角ID -- 位置编码即可区分。
4. 跨所有帧 token 的自注意力实现跨视角推理。
5. 模型学习跨相机角度关联信息。

输出 OUTPUT 统一隐变量序列: (B, 16, 7+, 28, 28)

- 所有相机隐变量沿时间维度拼接
- 位置: 帧 0-2 (cam1), 帧 3-4 (cam2), 帧 5-6 (cam3)
- 之后是动作/本体感知/未来帧
- 3D RoPE 位置编码区分每个视角的位置

RoboCasa 视角说明

agentview: 工作区俯视/正面视图

eye_in_hand: 安装在机械臂末端的腕部相机

robotface: 捕获机械臂构型的侧视图

不同时间长度因4倍时间压缩的帧填充导致。

CODE: cosmos_policy/experiments/robot/cosmos_utils.py:61 get_latent_indices_from_model_config() | tokenizers/wan2pt1.py:303 encode()

模块 18: 训练 vs 推理对比

Training vs Inference Comparison

训练 TRAINING

- 1. 从专家演示中收集 (obs, action, proprio, value)
- 2. VAE 编码所有观测 -> 隐变量
- 3. 将真实动作/本体感知注入隐帧
- 4. 创建条件掩码: obs=1, action/proprio=0
- 5. 对 mask=0 帧添加噪声: $x_sigma = x_0 + sigma * eps$
- 6. DiT 从含噪输入预测噪声 (或 x_0)
- 7. 损失 = MSE(预测, 目标), 仅在 mask=0 帧上计算
- 8. 混合 EDM: 70% 对数正态 + 30% 均匀 sigma 采样

推理 INFERENCE

- 1. 从机器人传感器采集当前观测
- 2. VAE 编码观测 -> 干净隐变量 (mask=1)
- 3. 初始化动作/本体感知帧为噪声 (mask=0)
- 4. 迭代去噪循环 (35步 ODE 求解器)
- 5. 每一步: DiT 从含噪输入预测 x_0
- 6. ODE 步将噪声向干净预测方向更新
- 7. 最终去噪隐变量包含预测的动作
- 8. 提取动作块, 执行前 N 步

核心区别

- 训练: 真实动作被注入隐帧 -> 模型学习去噪还原它们
- 推理: 动作帧从纯噪声开始 -> 模型通过迭代去噪生成它们
- 训练损失: 仅在 mask=0 帧上计算 MSE, 支持动作/价值分离的可选损失掩码
- 推理: 无需真实标签, 纯生成式预测, 从观测到动作的完整闭环

CODE: cosmos_policy/models/policy_text2world_model.py:310 compute_loss_with_epsilon_and_sigma() | :702 generate_samples_from_batch() | cosmos_policy/modules/cosmos_sampler.py:96 _forward_impl()